

[**Instructions:** Remove everything that is not a heading below and fill in with your own diagrams, etc.]

1. Brief introduction __/3

This document describes the Enemy AI System for a top-down stealth-action game. The AI system is responsible for controlling enemy awareness, movement behavior, and combat engagement across the game's levels. The AI operates using a state-based behavior model consisting of Stealth, Chase, and Attack states.

In Level 1 (Witch's House), the AI supports stealth-focused gameplay by reacting to sound and light triggers without allowing enemies to attack the player. Once the pineapple is collected, enemies become alert and begin chasing the player.

In Level 2 (Jungle Escape), the AI emphasizes pursuit and limited combat, allowing enemies to chase, search, and disengage based on player visibility.

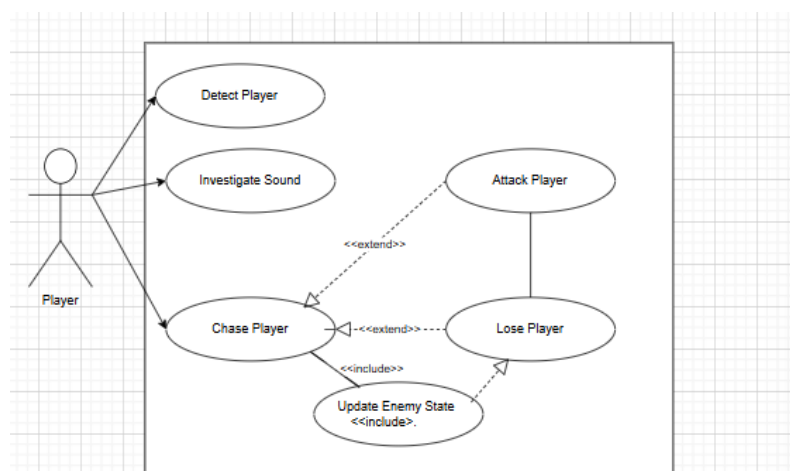
In Level 3 (Final Boss Arena), the AI immediately transitions into attack mode, enabling the boss fight mechanics.

The AI system integrates with the Level Design system for triggers and checkpoints and with the Boss Fight system for combat execution.

2. Use case diagram with scenario __14

Use Case Diagrams

Enemy AI System



Scenarios

Name: Detect Player

Summary: The Enemy AI detects the player based on sound, lighting, or proximity.

Actors: Player

Preconditions:

- Player is inside the level.
- Enemy AI is active.

Basic sequence:

Step 1: Enemy AI monitors player position.

Step 2: Enemy AI listens for sound cues (floor cracking).

Step 3: Enemy AI checks player visibility based on lighting.

Step 4: Enemy AI updates detection status.

Exceptions:

- Step 2: Player does not generate sound → Enemy AI remains idle.
- Step 3: Player stays in darkness → Detection fails.

Post conditions:

- Detection status is updated.

Priority: 1

ID: AI01

Name: Investigate Sound

Summary: The Enemy AI moves toward a sound source created by the player.

Actors: Player

Preconditions:

- Player generates a sound event.

Basic sequence:

Step 1: Enemy AI receives sound location.

Step 2: Enemy AI moves toward the sound source.

Exceptions:

- Step 1: Sound occurs outside detection radius → No response.

Post conditions:

- Enemy position is updated.

Priority: 2

ID: AI02

Name: Chase Player (*Triggered after Detect Player*)

Summary: The Enemy AI chases the player after detection.

Actors: Player

Preconditions:

- Player has been detected.

Basic sequence:

Step 1: Enemy AI switches to chase state.

Step 2: Enemy AI follows the player's position.

Exceptions:

- Step 2: Player escapes detection range → Extend Lose Player.

Post conditions:

- Enemy continues pursuit or exits chase state.

Priority: 1

ID: AI03

Name: Attack Player (<<extend>> Chase Player)

Summary: The Enemy AI attacks the player when within range.

Actors: Player

Preconditions:

- Player is within attack range.

Basic sequence:

Step 1: Enemy AI confirms attack range.

Step 2: Enemy AI executes attack behavior.

Exceptions:

- Step 1: Player moves out of range → Return to Chase Player.

Post conditions:

- Player health is reduced via combat system.

Priority: 1

ID: AI04

Name: Lose Player (<<extend>> Chase Player)

Summary: The Enemy AI loses the player and returns to stealth behavior.

Actors: Player

Preconditions:

- Enemy AI is chasing the player.

Basic sequence:

Step 1: Enemy AI checks visibility and sound input.

Step 2: Player is no longer detected.

Step 3: Enemy AI returns to patrol/stealth state.

Exceptions:

- Step 2: Player is re-detected → Return to Chase Player.

Post conditions:

- Enemy resumes stealth behavior.

Priority: 2

ID: AI05

Name: Update Enemy State (<<include>>)

Summary: The Enemy AI updates its internal state after behavior changes.

Actors: Enemy AI System

Preconditions:

- Enemy behavior changes.

Basic sequence:

Step 1: Enemy AI records current behavior state.

Step 2: Enemy AI saves updated state.

Exceptions:

- None.

Post conditions:

- Enemy state is stored for future logic.

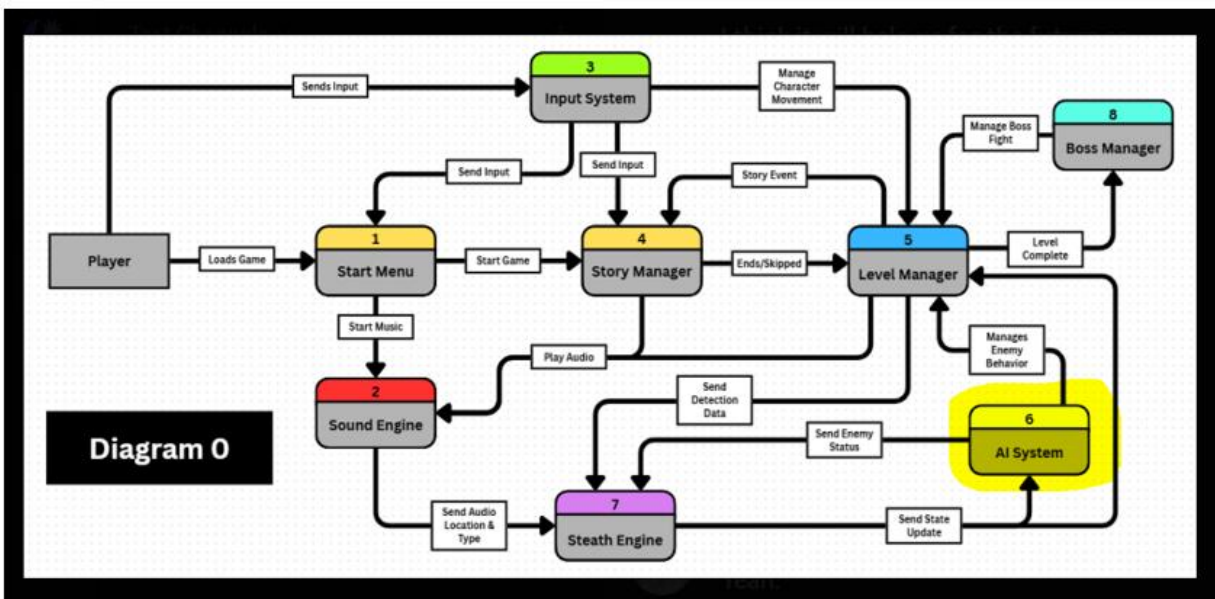
Priority: 2

ID: AI06

3. Data Flow diagram(s) from Level 0 to process description for your feature ____14

Data Flow Diagrams

Level 0 diagram



Process Descriptions

Processes (Enemy AI System)

3.1 Detect Player

Inputs:

- Player position
- Floor cracking sound
- Light exposure

Outputs:

- Detection status

3.2 Evaluate AI State

Inputs:

- Detection status
- Enemy state

Outputs:

- Stealth state
- Chase state
- Attack state

3.3 Execute Behavior

Inputs:

- AI state
- Player position

Outputs:

- Enemy movement
- Attack trigger

3.4 Update Enemy State

Inputs:

- Behavior result

Outputs:

- Updated enemy state

4. Acceptance Tests _____9

Enemy AI Feature

Each test evaluates a single Enemy AI rule using deterministic inputs and expected outputs. Tests are designed to be automatable using Unity's unit testing framework.

The output file will have the following characteristics:

- AI state is Stealth, Chase, or Attack
- No Attack in Level 1
- Chase above detection threshold
- Stealth below threshold
- Immediate Attack in boss arena
- Alert persists after pineapple

Unit Test Table — Enemy AI State Evaluation

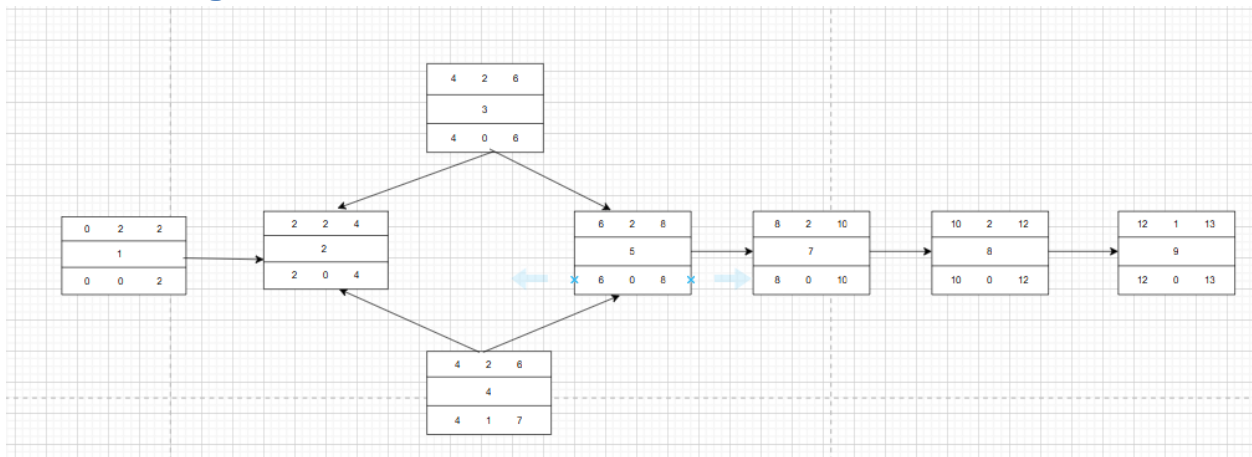
Test ID	Detection Value	Level	Pineapple Collected	Expected AI State	Notes
UT-01	0	Any	No	Stealth	No sound, no light
UT-02	Threshold - 1	Any	No	Stealth	Below boundary
UT-03	Threshold	Any	No	Chase	At boundary
UT-04	Threshold + 1	Any	No	Chase	Above boundary
UT-05	Threshold + 1	Level 1	No	Chase	Attack disabled
UT-06	Threshold + 1	Level 2	No	Attack	Combat enabled
UT-07	Any	Boss Arena	Any	Attack	Immediate attack
UT-08	-1	Any	No	Stealth	Invalid input clamped
UT-09	Max + 1	Any	No	Chase	Invalid input clamped
UT-10	Any	Any	Yes	Chase	Alert persists
UT-11	Any	Any	No	Stealth	Respawn without pineapple

5. Timeline ____/10

Work items- Enemy Ai System

Task	Duration (PWks)	Predecessor Task(s)
1. AI Requirements Analysis	2	-
2. Detection Logic Design (Sound & Light)	3	1
3. Stealth Behavior Implementation	2	2
4. Chase Behavior Implementation	2	2
5. Attack Behavior Implementation	2	3,4
6. Alert State Persistence (Pineapple Logic)	1	3
7. Boss Fight AI Integration	2	5
8. AI Testing & Debugging	2	6,7
9. Documentation	1	8

Pert diagram



Gantt timeline

